



Komplexitätstheorie

NP-Vollständigkeit

RECAP: Komplexitätstheorie

- Elemente der Klasse NP sind Entscheidungsprobleme
- Wir können Entscheidungsprobleme und Optimierungsprobleme ineinander überführen.
- Bekannte Vertreter von NP Problemen sind
 - Travelling Salesman, das Bin Packing oder das Knapsack Problem
- Der Nachweis, dass ein Problem in NP liegt, ist meist recht einfach:
 - Rate ein Zertifikat und verifiziere es.
- Es ist nicht bewiesen, dass P eine echte Teilmenge von NP ist.
 - Alle Probleme in P liegen auch in NP.
 - TSP oder andere schwere Probleme in NP könnten auch in P liegen.

RECAP: NP Vollständigkeit

- L_1 und L_2 seien zwei Sprachen über Σ_1 bzw. Σ_2 . Dann ist L_1 **polynomiell reduzierbar** auf L_2 , wenn es eine Reduktion von L_1 nach L_2 gibt, die in polynomieller Zeit berechenbar ist.
- Wir schreiben $L_1 \leq_p L_2$.
- Erinnerung: Eine Reduktion $L_1 \leq L_2$ ist definiert durch eine berechenbare Funktion $f: \Sigma_1^* \rightarrow \Sigma_2^*$, so dass für all $x \in \Sigma_1^*$ gilt: $x \in L_1 \Leftrightarrow f(x) \in L_2$.
- Bei der **Polynomialzeitreduktion** $L_1 \leq_p L_2$ verlangen wir also lediglich zusätzlich, dass f in polynomieller Zeit berechnet werden kann.
- Intuitiv: $L_1 \leq_p L_2$ bedeutet „ L_1 ist nicht schwieriger als L_2 “, modulo polynomieller Eingabetransformation.

RECAP: NP-harte Probleme

Ein Problem L heißt **NP-hart**, falls gilt $\forall L' \in \text{NP}: L' \leq_p L$.
 (Im Englischen NP-hard, man findet auch die Übersetzung NP-schwer.)

Satz: Sei $L \in P$ und L NP-hart $\Rightarrow P = \text{NP}$.

Beweis:

Sei L' ein beliebiges Problem aus NP. Da L NP-hart ist, gilt $L' \leq_p L$.

Aus $L \in P$ folgt wegen des Lemmas, dass $L' \in P$.

Also ist jedes Problem aus NP auch in P enthalten, so dass gilt $P = \text{NP}$.

QED

Unter der Hypothese $P \neq \text{NP}$ gibt es für NP-harte Probleme deshalb keinen Polynomialzeitalgorithmus.

RECAP: NP Vollständigkeit

- Ein Problem L heißt **NP-vollständig** (NP-complete), falls gilt $L \in \text{NP}$ und L ist NP-hart.
 - Intuitiv: dies sind die schwierigsten unter den Problemen in NP.
- NPC bezeichnet die Klasse der **NP-vollständigen Probleme**.
 - Alle Probleme in NPC können als gleich schwierig modulo polynomieller Eingabetransformationen angesehen werden.

RECAP: NP-hart vs. NP-vollständig

- Worin liegt der Unterschied zwischen NP-hart und NP-vollständig?
- NP-hart
Ein Problem L heißt **NP-hart**, falls gilt $\forall L' \in \text{NP}: L' \leq_p L$.
- NP-vollständig
Ein Problem L heißt **NP-vollständig** (NP-complete), falls gilt $L \in \text{NP}$ und L ist NP-hart.



SAT ist NP-vollständig

Der Satz von Cook und Levin

Gibt es NP-vollständige Problem?

- Der erste NP-Vollständigkeitsbeweis wurde unabhängig von Stephen Cook (1971) und Leonid Levin (Veröffentlichung: 1973) geführt.
- **Master-Reduktion:** alle Probleme aus NP können in polynomieller Zeit auf das Erfüllbarkeitsproblem SAT für aussagenlogische Formeln reduziert werden.
- Richard Karp (1972) hat demonstriert, dass man diese NP-Härte von SAT dazu verwenden kann, um für andere Probleme relativ leicht nachzuweisen, dass sie NP-hart sind.
- Bis heute die wichtigste Methode um zu zeigen, dass ein Problem zumindest unter $P \neq NP$ keinen Polynomialzeitalgorithmus zulässt.

Der Satz von Cook und Levin

Problem (**Erfüllbarkeitsproblem, Satisfiability, SAT**)

- Eingabe: Aussagenlogische Formel ϕ in CNF
- Frage: Gibt es eine erfüllende Belegung für ϕ ?

Erinnerung: ϕ ist in CNF, wenn es aus $\wedge$ -verknüpften Klauseln besteht, die wiederum aus $\vee$ -verknüpften Literalen bestehen. Ein Literal entspricht dabei einer Variablen x_i oder ihrer Negierung $\bar{x}_i$.

Satz (Cook, Levin; Masterreduktion):

SAT ist NP-vollständig.

Der Satz von Cook und Levin

Beweis:

Eine aussagenlogische Formel ϕ kann bezüglich einer gegebenen Belegung in polynomieller Zeit ausgewertet werden.

Somit können wir eine erfüllende Belegung als Zertifikat einsetzen. Die Länge dieses Zertifikates ist linear (und damit polynomiell) in der Eingabelänge beschränkt.

Es gilt also **SAT $\in$ NP**.

Der schwierige Teil des Beweises: zeige NP-Härte von SAT. Reduziere dazu jedes Problem aus NP polynomiell auf SAT (Master-Reduktion).

Der Satz von Cook und Levin

Beweis (cont.):

Sei $L \subseteq \Sigma^*$ ein beliebiges Problem aus NP. Für jedes $x \in \Sigma^*$ müssen wir in polynomieller Zeit eine Formel ϕ konstruieren, die die folgende Eigenschaft erfüllt.

$$x \in L \Leftrightarrow \phi \in \text{SAT}$$

Informationen über L :

- Entscheidungsproblem mit NTM M , die L in polynomieller Zeit erkennt.
- Laufzeitschranke von M ist durch Polynom p in der Eingabelänge beschrieben.

Beschreibe einen Polynomialzeitalgorithmus, der für jedes $x \in \Sigma^*$ eine aussagenlogische Formel ϕ konstruiert mit der Eigenschaft:

$$M \text{ akzeptiert } x \Leftrightarrow \phi \text{ ist erfüllbar}$$

Der Satz von Cook und Levin

Beweis (cont.):

- Die Zustandsmenge der NTM M bezeichnen wir wie üblich mit Q , das Bandalphabet mit Γ und die Überföhrungsrelation mit δ .
- OBdA (Vereinfachungen wurden in der Übung bewiesen).
 - Die NTM M bewegt den Kopf niemals an eine Position links der Startposition.
 - Die Zustandsmenge Q enthalte genau einen akzeptierenden Zustand q_{accept} .
 - Wenn q_{accept} erreicht wird, so terminiert M nicht, sondern verbleibt in einer Endlosschleife im Zustand q_{accept} .

Der Satz von Cook und Levin

Beweis (cont.):

Sei $x \in \Sigma^*$ eine beliebige Eingabe, $n = |x|$ und $K_0 = q_0x$ die Startkonfiguration von M .

Ziel: Konstruktion einer Formel ϕ , die genau dann erfüllbar ist, wenn es eine mögliche Konfigurationsfolge $K_0 \vdash K_1 \vdash \dots \vdash K_{p(n)}$ für M auf Eingabe x gibt, bei der $K_{p(n)}$ eine Konfiguration im Zustand q_{accept} ist.

Dazu: definiere geeignete Menge von booleschen Variablen, die es uns ermöglicht, einzelne Konfigurationen und Konfigurationsübergänge in einer aussagenlogischen Formel zu kodieren.

Wir vernachlässigen dabei zunächst, dass die Formel in CNF vorliegen soll.

Der Satz von Cook und Levin

Beweis (cont.):

Definition von drei Arten von **Variablen** für Zustand, Kopfposition und Bandinschrift zu jedem Zeitpunkt innerhalb der Laufzeitschranke

Beachte: Laufzeitschranke für M ist auch eine Schranke für die maximal erreichbare Kopfposition. Betrachte Kopfpositionen und Bandinschriften von Index 0 bis Index $p(n)$

- $Q(t, k)$ für $t \in \{0, \dots, p(n)\}$ und $k \in Q$
 - $Q(t, k) = 1$, wenn die Rechnung zum Zeitpunkt t in Zustand k , sonst $Q(t, k) = 0$
- $H(t, j)$ für $t, j \in \{0, \dots, p(n)\}$
 - $H(t, j) = 1$, wenn die Rechnung zum Zeitpunkt t an Bandposition j befindet, sonst $H(t, j) = 0$
- $S(t, j, a)$ für $t, j \in \{0, \dots, p(n)\}$ und $a \in \Gamma$
 - $S(t, j, a) = 1$, wenn zum Zeitpunkt t an Bandposition j das Zeichen a steht, sonst $S(t, j, a) = 0$.

Da wir nur an polynomiell vielen Rechenschritten und daher nur an polynomiell vielen Bandpositionen interessiert sind, benötigen wir auch nur eine polynomiell beschränkte Anzahl an Variablen.

Der Satz von Cook und Levin

Beweis (cont.):

Kodierung der Konfigurationen:

Für jedes $t \in \{0, \dots, p(n)\}$, beschreiben wir eine Formel ϕ_t für $t \in \{0, \dots, p(n)\}$, die genau dann erfüllt ist, wenn die Belegung $H(t, j)$, $Q(t, j)$, $S(t, j, a)$ zum Zeitpunkt t eine korrekte Konfiguration beschreibt:

- Es gibt genau eine Bandposition $j \in \{0, \dots, p(n)\}$ mit $H(t, j) = 1$
- Es gibt genau einen Zustand $k \in Q$ mit $Q(t, k) = 1$
- Für jedes $j \in \{0, \dots, p(n)\}$ gibt es genau ein Zeichen $a \in \Gamma$ mit $S(t, j, a) = 1$

Die Forderung von „genau eins“ können wir durch eine Formel in quadratischer/kubischer Länge darstellen. Die obigen Anforderungen, dass es nur einen Zustand, eine Bandposition und ein Zeichen je Bandposition gibt, können wir also jeweils durch eine Formel mit polynomiell beschränkter Länge kodieren.

Die Teilformeln für die oben gestellten Anforderungen fassen wir in der Formel ϕ_t zusammen und verknüpfen sie mit $\wedge$. Offensichtlich kann die so entstehende Formel ϕ_t durch einen Polynomialzeitalgorithmus berechnet werden.

Der Satz von Cook und Levin

Beweis (cont.):

Kodierung von Konfigurationsübergängen.

Wir betrachten nun nur noch Belegungen, die die Formeln $\phi_0, \dots, \phi_{p(n)}$ erfüllen und somit Konfigurationen $K_0, \dots, K_{p(n)}$ beschreiben, allerdings nicht notwendigerweise in einer Konfigurationsfolge der NTM M .

Als nächstes (2 Teile) konstruieren wir eine Formel ϕ'_t für $1 \leq t \leq p(n)$, die nur durch solche Belegungen erfüllt wird, wenn $K_{t-1} \vdash K_t$.

(Teil 1) Lege zuerst fest, dass die Bandinschrift von K_t an allen Positionen außer der Kopfposition zum Zeitpunkt $t - 1$ mit der Inschrift von K_{t-1} übereinstimmt, da sich beim Konfigurationsübergang die Bandinschrift ja nur an der Kopfposition verändern darf:

$$\bigwedge_{i=0}^{p(n)} \bigwedge_{z \in \Gamma} ((S(t-1, i, z) \wedge \overline{H(t-1, i)}) \Rightarrow S(t, i, z))$$

Erklärt von links nach rechts liest: "Für alle Bandpositionen i und alle Zeichen z gelte, falls zum Zeitpunkt $t - 1$ das Band an Position i das Zeichen z enthält und die Kopfposition nicht i ist, so enthalte das Band auch zum Zeitpunkt t an Position i das Zeichen z ."

Der Ausdruck $A \Rightarrow B$ kann durch den äquivalenten Ausdruck $\overline{A} \vee B$ ersetzt werden.

Der Satz von Cook und Levin

Beweis (cont.):

Kodierung von Konfigurationsübergängen (cont.).

(Teil 2) Überführungsrelation δ beschreibt erlaubte Veränderungen beim Konfigurationsübergang. Die Kopfbewegung implementieren wir durch einen Term $\kappa \in \{-1, 0, 1\}$, welcher auf die Kopfposition aufaddieren wird, d.h. $L \equiv -1, R \equiv 1, N \equiv 0$.

Die folgende Formel beschreibt die Veränderungen, die beim Übergang von K_{t-1} nach K_t erlaubt sind, falls sich die Rechnung zum Zeitpunkt $t - 1$ im Zustand k befindet und der Kopf an Position j das Zeichen a liest.

$$(Q(t-1, k) \wedge H(t-1, j) \wedge S(t-1, j, a)) \Rightarrow \bigvee_{(k, a, k', a', \kappa) \in \delta} (Q(t, k') \wedge H(t, j + \kappa) \wedge S(t, j, a'))$$

Erklärung: „Wenn zum Zeitpunkt $t - 1$ die Rechnung im Zustand k ist, der Kopf an Position j ist, und an dieser Position das Zeichen a steht, so führe in Schritt t eine mögliche δ -Überführungen bezüglich Zustand, Position und geschriebenem Zeichen durch.“

(Zusammenfassung) Diese Formel wird für alle $k \in Q, j \in \{0, \dots, p(n)\}$ und $a \in \Gamma$ benötigt. Die $\wedge$ -Verknüpfung der beiden oben beschriebenen Teilformeln ergibt die Formel ϕ'_t , die einen korrekten Konfigurationsübergang von K_{t-1} nach K_t sicherstellt.

Diese Formel hat polynomielle Länge und kann durch einen Polynomialzeitalgorithmus berechnet werden.

Der Satz von Cook und Levin

Beweis (cont.):

Zusammensetzen der Formel.

Zu den oben beschriebenen Teilformeln ϕ_t und ϕ'_t fügen wir noch weitere einfache Teilformeln hinzu, die festlegen, dass die Rechnung in der Startkonfiguration $K_0 = q_0x$ startet und nach $p(n)$ Schritten im Zustand q_{accept} befindet:

$$\phi = Q(0, q_0) \wedge H(0, 0) \wedge \bigwedge_{i=0}^{n-1} S(0, i, x_i) \wedge \bigwedge_{i=n}^{p(n)} S(0, i, B) \wedge \bigwedge_{i=0}^{p(n)} \phi_i \wedge \bigwedge_{i=1}^{p(n)} \phi'_i \wedge Q(p(n), q_{accept})$$

Formel ϕ derart konstruiert, dass jede akzeptierende Konfigurationsfolge von M genau einer erfüllenden Belegung von ϕ entspricht. Somit ist ϕ genau dann erfüllbar, wenn eine akzeptierende Konfigurationsfolge von M auf x existiert, also im Falle $x \in L$.

Wir haben bereits argumentiert, dass ϕ in polynomieller Zeit aus x konstruiert werden kann.

Desweiteren kann ϕ in polynomieller Zeit in CNF überführt werden.

QED

NP-vollständige Probleme

- Cook (1971): SAT NP vollständig
- Karp (1972):
 - 21 Probleme NP vollständig durch Rückführen auf SAT
 - Bis heute die wichtigste Methode um zu zeigen, dass ein Problem zumindest unter $P \neq NP$ keinen Polynomialzeitalgorithmus zulässt.

Konsequenzen

Konsequenzen aus dem Satz von Cook:

- Da SAT in exponentieller Zeit $2^{O(n)}$ entscheidbar ist (Wahrheitstafel, alle Belegungen durchprobieren) und alle NP-Probleme polynomial auf SAT reduzierbar sind, sind damit auf jeden Fall alle NP-Probleme in der Zeit $O(2^{p(n)})$, für ein Polynom p , entscheidbar.
- Es gibt Erfüllbarkeits-Checker für aussagenlogische Formeln in konjunktiver Normalform (sogenannte SAT-Solver), die in der Praxis gut funktionieren.
- Für bestimmte Formeln zeigen sie jedoch ein schlechtes Laufzeitverhalten.
- Diese SAT-Solver kann man über Reduktionen nutzen, um damit auch andere NP-vollständige Probleme in der Praxis zu lösen.

NP-vollständige Probleme

Karp:

- Um zu zeigen, dass L NP-vollständig ist, muss $L \in \text{NP}$ und L muss NP-hart sein
- Die Zugehörigkeit zu NP meist leicht durch geeignetes Zertifikat zu beweisen
- Schwierigkeit: NP-Härte zu beweisen.
 - Masterreduktion nur einmal nötig (SAT)
 - Statt dessen reduziere NP-hartes Problem, anfangend mit SAT, auf andere Probleme.

Lemma: Sei L^* NP-hart und $L^* \leq_p L$. Dann ist L NP-hart.

Beweis: Gemäß Voraussetzung gilt $\forall L' \in \text{NP}: L' \leq_p L^*$ und $L^* \leq_p L$. Aufgrund der Transitivität der polynomiellen Reduktion folgt somit $\forall L' \in \text{NP}: L' \leq_p L$.

QED

NP-vollständige Probleme

SAT ist Ausgangspunkt für weitere Härtebeweise.

Bei einer Formel in 3CNF enthalten alle Klauseln genau drei Literale.

Wenn wir SAT auf aussagenlogische Formeln dieser Form beschränken, nennen wir das Problem 3SAT.

Problem (**3SAT**):

- Eingabe: Aussagenlogische Formel ϕ in 3CNF
- Frage: Gibt es eine erfüllende Belegung für ϕ ?

NP-vollständige Probleme : 3SAT

- 3SAT ist ein Spezialfall von SAT und deshalb wie SAT in NP.
- Um zu zeigen, dass 3SAT ebenfalls NP-vollständig ist, müssen wir also "nur" noch die NP-Härte von 3SAT nachweisen.

Lemma $SAT \leq_p 3SAT$.

Beweis:

Gegeben sei eine Formel ϕ in CNF. Wir müssen beschreiben, wie ϕ in polynomieller Zeit in eine erfüllbarkeitsäquivalente Formel ϕ' in 3CNF überführt werden kann, d.h. ϕ ist erfüllbar $\Leftrightarrow \phi'$ ist erfüllbar.

NP-vollständige Probleme : 3SAT

Beweis (cont.):

- Diejenigen Klauseln aus ϕ , in denen weniger als drei Literale vorkommen, können wir einfach in logisch äquivalente Klauseln mit drei Literalen umformen, indem wir eines der Literale in der Klausel wiederholen.
- Sei nun $C = l_1 \vee l_2 \vee \dots \vee l_k$ eine Klausel aus ϕ mit $k > 3$ Variablen, wobei l_i jeweils für ein negiertes oder unnegiertes Literal steht. In einer **Klauseltransformation** ersetzen wir C durch die Teilformel $C = C' \wedge C''$ mit $C' = l_1 \vee l_2 \vee \dots \vee l_{k-2} \vee h$ und $C'' = \bar{h} \vee l_{k-1} \vee l_k$, wobei h eine zusätzlich eingeführte Hilfsvariable bezeichnet. Aus der Klausel C der Länge $k > 3$ haben wir somit eine Klausel C' der Länge $k - 1$ und eine Klausel C'' der Länge drei erzeugt.
- Diese Art der Klauseltransformation wenden wir solange auf ϕ an, bis eine Formel ϕ' in 3CNF vorliegt, wobei wir in jeder Klauseltransformation eine neue Hilfsvariable erzeugen.
- Um die Korrektheit unserer Konstruktion nachzuweisen, müssen wir zeigen, dass jede einzelne Klauseltransformation die Erfüllbarkeitsäquivalenz bewahrt.

NP-vollständige Probleme: 3SAT

Beweis (cont.):

- Jede einzelne Klauseltransformation kann in Zeit polynomiell in der Klausellänge durchgeführt werden.
- Für jede Klausel der Länge k in der Formel ϕ benötigen wir $k - 2$ Transformationen. Die Gesamtanzahl der benötigten Klauseltransformationen ist also durch die Anzahl der in ϕ vorkommenden Literale nach oben beschränkt.
- Die Reduktion kann somit in polynomieller Zeit berechnet werden.

QED

NP-vollständige Probleme: 3SAT

Das Lemma ergibt das folgende Ergebnis:

Satz: 3SAT ist NP-vollständig.

- Die Reduktion von SAT auf 3SAT ist nicht besonders spektakulär, da die beiden Probleme offensichtlich miteinander verwandt sind.
- Die eigentliche Stärke der Reduktionstechnik liegt darin, dass sie Probleme aus unterschiedlichsten Bereichen zueinander in Beziehung setzen kann.
- Im Folgenden veranschaulichen wir dies, indem wir 3SAT nun auf verschiedene Graph- und Zahlprobleme reduzieren und damit die NP-Vollständigkeit dieser Probleme nachweisen.

NP-vollständige Probleme: CLIQUE

- Da wir schon wissen, dass das Cliquenproblem in NP ist, müssen wir zum Nachweis der NP-Vollständigkeit nur noch die NP-Härte nachweisen. Dazu führen wir eine Polynomialzeitreduktion von 3SAT auf CLIQUE durch

Satz: CLIQUE ist NP-vollständig.

Beweis:

- Gegeben sei eine 3KNF-Formel ϕ .
- Wir beschreiben eine Transformation von ϕ in einen Graphen $G = (V, E)$ und eine Zahl $k \in \mathbb{N}$ mit der Eigenschaft :

ϕ ist erfüllbar $\Leftrightarrow G$ hat eine k -Clique

Also: $3SAT \leq_p CLIQUE$

NP-vollständige Probleme: CLIQUE

Beweis (cont.):

- Seien $C_1, \dots, C_m$ die Klauseln von ϕ und $\ell_{i,1}, \ell_{i,2}, \ell_{i,3}$ die Literale in der Klausel C_i .
- Wir identifizieren die Knoten des Graphen mit den Literalen der Formel, d.h. wir setzen $V = \{\ell_{i,j} | 1 \leq i \leq m, 1 \leq j \leq 3\}$.
- Die Kantenmenge E definieren wir wie folgt. Ein Knotenpaar wird durch eine Kante verbunden, es sei denn
 - 1) die mit den Knoten assoziierten Literale gehören zur selben Klausel oder
 - 2) eines der beiden Literale ist die Negierung des anderen Literals.
- Den Parameter k des Cliquenproblems setzen wir gleich m , also gleich der Anzahl der Klauseln in ϕ .
- Diese einfache Eingabetransformation kann offensichtlich in polynomieller Zeit berechnet werden.

NP-vollständige Probleme: CLIQUE

Beweis (cont.):

- Zeige noch: der Graph G hat **genau dann** eine k -Clique, wenn die Formel ϕ erfüllbar ist.
- Nehme zunächst an, ϕ hat eine erfüllende Belegung.
- Diese Belegung erfüllt in jeder Klausel mindestens ein Literal.
- Pro Klausel wählen wir eines dieser erfüllten Literale beliebig aus.
- Zeige, die zugehörigen $k = m$ Knoten bilden eine Clique in G .
- Alle selektierten Literale gehören zu verschiedenen Klauseln, so dass keine Kante aufgrund von Ausnahmeregel 1 fehlt.
- Alle selektierten Literale werden gleichzeitig erfüllt, so dass kein Paar von sich widersprechenden Literalen dabei ist und keine Kante wegen Ausnahmeregel 2 fehlt.
- Also sind alle selektierten Knoten verbunden
- Also: G enthält eine k -Clique.

NP-vollständige Probleme: CLIQUE

Beweis (cont.):

- Jetzt nehmen wir an G enthält eine k -Clique.
- Aufgrund von Ausnahmeregel 1 müssen die Knoten in dieser Clique zu verschiedenen Klauseln gehören.
- Aus $k = m$ folgt somit, dass die k -Clique genau ein Literal pro Klausel selektiert.
- Diese Literale können alle gleichzeitig erfüllt werden, da sie sich wegen Ausnahmeregel 2 nicht widersprechen.
- Also ist ϕ erfüllbar.

QED